

LEARNING RESOURCE GUIDE

ISC2 CCSP: Certified Cloud Security Professional (Aug 2026) | Professional Participant Workbook

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- By the end of this lecture, you will be able to explain the structural differences between Waterfall and Agile SDLC methodologies and the security implications of each.
- By the end of this lecture, you will be able to identify the security strengths and weaknesses of Waterfall's sequential phase-gate model.
- By the end of this lecture, you will be able to describe how security is embedded in Agile development through backlog management, sprint-level threat modelling, and automated pipeline controls.
- By the end of this lecture, you will be able to explain the risk of sprint pressure and security debt in Agile teams, and identify the practice that prevents it.
- By the end of this lecture, you will be able to apply the principle that shifting security left is the economically rational choice, regardless of methodology.

Introduction

Every software development methodology is an attempt to answer the same question: in what order should we do the work, and how do we know when each stage is complete? The answer to that question has deep implications for security, because security is not a single activity — it is a set of activities that must be placed at the right moment in the development cycle to be effective. Two methodologies dominate enterprise development: Waterfall, which answers the question with sequential phases and formal gates, and Agile, which answers it with short iterative cycles and continuous delivery. Both can produce secure software. Both can fail. The determining factor is always whether security is built into the methodology from the beginning.

The CCSP exam tests candidates on both methodologies in a security context, and the framing that matters most is cost. Research across software engineering consistently shows that a defect found during the requirements or design phase costs a fraction of what the same defect costs to fix after deployment. This is the economic argument for shifting security left — moving security activities earlier in the development process. Waterfall shifts security left through formal phase gates that require security review before the next phase begins. Agile shifts security left by embedding security into every sprint rather than treating it as a final-phase audit.

For Cloud Security Practitioners, the methodology question is rarely theoretical. You will encounter teams using Waterfall in regulated industries where documentation is a compliance requirement, and teams using Agile in cloud-native environments where continuous delivery is a business expectation. Your role is to understand how security integrates with each methodology and to advocate for the practices that keep security from being deferred — regardless of which methodology the team has chosen.

Core Concepts Explained

Waterfall: Sequential Phases and Security Gate Model

Definition: Waterfall is a sequential development methodology in which each phase — requirements, design, development, testing, deployment — must be completed before the next begins. Each phase produces formal output artefacts that serve as the input for the next phase, creating natural security checkpoints.

Business relevance: Waterfall's sequential structure is well-suited to regulated industries such as defence, healthcare, and financial services, where formal documentation of security requirements, design decisions, and test results is a compliance requirement.

Cloud Security Practitioner relevance: Cloud Security Practitioners in Waterfall projects participate in each phase's security gate: reviewing and approving the requirements specification for completeness of security requirements, participating in architecture reviews, reviewing static analysis and test results, and signing off on deployment configurations.

Real-world example: A defence contractor developing a classified data processing application for a government agency used a Waterfall methodology aligned to the NIST Risk Management Framework. During the requirements phase, the security architect produced a complete System Security Plan (SSP) that documented every NIST SP 800-53 control applicable to the system. During the design phase, the SSP was used to evaluate the proposed architecture against each control. During testing, a formal Security Assessment was conducted by an independent assessor. This documentation trail — SSP, architecture approval, security assessment — was required for the Authority to Operate (ATO) that authorised the system to process classified data. Without the Waterfall phase structure and its formal artefacts, the ATO process would not have had the documentation it required.

Risks or limitations: Waterfall's greatest risk is time-gap vulnerability: a flaw introduced in the design phase may not be discovered until integration testing months later. The longer the gap between introduction and discovery, the more expensive the fix. The best practice to mitigate this is introducing security checkpoints within phases — not only at phase boundaries — so that design flaws are caught before they are fully implemented.

Waterfall Security Strengths: Documentation and Audit Trail

Definition: Waterfall projects generate detailed specifications, data flow diagrams, threat models, and design records that security teams can review as formal artefacts. This documentation has direct compliance value in regulated industries.

Business relevance: In a PCI DSS audit, a Waterfall project can point to documented security requirements from the requirements phase, approved architecture diagrams from the design phase, and formal penetration test results from the testing phase. That audit trail significantly reduces the time and cost of demonstrating compliance.

Cloud Security Practitioner relevance: Cloud Security Practitioners in Waterfall environments are responsible for ensuring that security artefacts produced at each phase are complete, accurate, and stored in an accessible repository — because incomplete artefacts undermine the compliance value of the entire methodology.

Real-world example: A UK financial services firm undergoing a PCI DSS Level 1 assessment for a new payment processing application built on AWS presented the assessor with a complete set of Waterfall phase artefacts: a 45-page security requirements document from the requirements phase, an annotated data flow diagram with threat modelling annotations from the design phase, OWASP ASVS Level 2 compliance evidence from the implementation phase, and a formal penetration test report from the testing phase. The assessor was able to

trace every PCI DSS Requirement to a specific control documented in one of these artefacts. The firm's Qualified Security Assessor commented that this was the most complete pre-assessment documentation package they had reviewed that year. The Waterfall structure, and the security team's discipline in producing complete artefacts at each phase, directly reduced the audit duration and cost.

Risks or limitations: The risk of relying solely on Waterfall's documentation model is that documentation completeness becomes a proxy for security quality. A well-documented insecure design is still an insecure design. The best practice is treating documentation as evidence of security thinking, not a substitute for it.

Agile Security: Embedding Controls in the Development Rhythm

Definition: Agile breaks development into short iterations (sprints) of two to four weeks. Security is embedded in the development rhythm by placing security requirements in the backlog, conducting sprint-level threat modelling, and running automated security scans in the CI/CD pipeline.

Business relevance: Agile's iterative model means security vulnerabilities introduced in a sprint can be detected and remediated within hours — if the right automated tools are in place — rather than months later as in a Waterfall testing phase.

Cloud Security Practitioner relevance: Cloud Security Practitioners in Agile environments maintain the security items in the backlog, define and enforce the security criteria in the definition of done, configure and monitor automated security tooling in the CI/CD pipeline, and participate in sprint ceremonies to keep security visible and prioritised.

Real-world example: Microsoft's internal security transformation in the early 2000s demonstrated what embedded Agile security looks like at scale. Facing significant criticism of the security quality of Windows and Internet Explorer, Microsoft halted Windows development for two months in 2002 to retrain every developer on security, then embedded security requirements, threat modelling, and automated testing into the development process itself rather than treating them as a post-development audit. In subsequent product releases, Microsoft documented measurable reductions in the number of critical vulnerabilities discovered in shipped products. The principle — when security is part of the development rhythm rather than a separate review, defect rates drop — has held as a foundation of modern secure development practice.

Risks or limitations: The primary risk in Agile is sprint pressure. When teams race to deliver features by a sprint deadline, security tasks can be deferred to the next sprint — then the next. Each deferral feels small. The cumulative result is growing security debt that does not surface until a penetration test or, worse, a breach. The best practice that prevents this is making security a blocking criterion in the definition of done: a sprint is not marked complete until security checks have passed.

Definition of Done as a Security Control

Definition: In Agile, the 'definition of done' is the agreed set of criteria that must be satisfied before a sprint or user story is considered complete. Including security criteria in the definition of done makes security a blocking requirement rather than an optional activity.

Business relevance: A definition of done that includes security prevents the common failure pattern in which security is deferred sprint after sprint under deadline pressure. It creates an organisational norm that security is non-negotiable, not a quality-of-life enhancement.

Cloud Security Practitioner relevance: Cloud Security Practitioners negotiate the security criteria in the definition of done with the product owner and development team at the start of the project, and review sprint results to confirm the criteria were actually satisfied rather than checked as a formality.

Real-world example: An Agile team building a cloud-native lending platform included three security criteria in their definition of done: all code must pass the Semgrep static analysis scan with no high-severity findings, all

new API endpoints must have corresponding automated security test cases, and any new external dependency must be reviewed and approved by the security lead. In the fourth sprint, a developer introduced a new open-source PDF generation library without the security review. The CI pipeline flagged the missing approval, the sprint was not marked done, and the security lead reviewed the library within the day — identifying that its default configuration wrote temporary files to a world-readable directory, a finding that was patched before the library reached production. The definition of done criteria directly prevented a vulnerability from shipping.

Risks or limitations: The risk of omitting security from the definition of done is that security becomes voluntary — teams that are ahead of schedule may include it, teams under pressure will not. The best practice is making at least one automated security gate a hard blocker in the CI pipeline so that the definition of done cannot be bypassed even under deadline pressure.

Cloud-Native Security in Agile Delivery Pipelines

Definition: Cloud application teams can provision infrastructure, integrate services, and deploy code at a pace Waterfall planning never anticipated. Security in cloud Agile delivery requires infrastructure-as-code scanning, automated compliance checks, and cloud-native security tooling embedded in the CI/CD pipeline.

Business relevance: Without automated security gates in the delivery pipeline, the speed advantage of cloud Agile development becomes a security liability. Every sprint that deploys to the cloud without automated checks is an opportunity for misconfiguration to reach production.

Cloud Security Practitioner relevance: Cloud Security Practitioners design the security architecture of the CI/CD pipeline — selecting and configuring static analysis tools, infrastructure-as-code scanners, container image scanners, and runtime monitoring tools — and defining the policies that determine when a pipeline run is blocked versus allowed to proceed.

Real-world example: A retail organisation running a cloud-native e-commerce platform on Azure integrated three automated security tools into their GitHub Actions pipeline: Checkov for Terraform infrastructure-as-code scanning (blocking on critical misconfigurations such as public storage accounts and unencrypted managed disks), Trivy for container image scanning (blocking on critical CVEs in base images), and OWASP ZAP for dynamic application scanning against the staging environment (blocking on OWASP Top 10 findings). In a six-month period, these tools blocked 23 deployments that would have introduced critical security issues — including three instances where a developer had accidentally configured a storage account as publicly accessible, reproducing exactly the misconfiguration pattern that had caused the lecture's example breach.

Risks or limitations: The risk of poorly configured pipeline security tools is alert fatigue — too many false positives cause teams to disable or ignore tools. The best practice is starting with a narrow set of high-confidence, high-severity rules and expanding coverage gradually, rather than enabling all available checks on day one.

Practical Applications

Understanding how Waterfall and Agile handle security gives Cloud Security Practitioners the vocabulary to advocate for the right practices in any development environment they encounter.

Security Gate Review in a Waterfall Project

A Cloud Security Practitioner assigned to a Waterfall project developing a regulated financial application on Google Cloud Platform establishes a security review checkpoint at the boundary of each SDLC phase. At the requirements phase exit, they review the requirements specification for completeness of security requirements, confirming that every applicable PCI DSS Requirement is addressed by a testable acceptance criterion. They

produce a sign-off document that the project manager cannot bypass on the way to the design phase. At the design phase exit, they participate in the architecture review, reviewing the data flow diagram against the STRIDE threat categories and confirming that the proposed controls address the prioritised threats. They annotate the design document with threat modelling findings and sign off on the architecture before development begins. At the testing phase exit, they review the penetration test report and confirm that all critical and high findings have remediation plans before deployment is authorised. Each phase exit produces a security artefact — the requirements sign-off, the annotated architecture, the penetration test review — that forms the compliance evidence package for the application's PCI DSS assessment.

Embedding Security in an Agile Sprint Cycle

A Cloud Security Practitioner working with an Agile cloud development team negotiates the security criteria in the definition of done before the first sprint begins. They agree with the product owner and lead developer on three blocking criteria: all commits must pass Semgrep static analysis with no high-severity findings before merging to the main branch, all new external dependencies must have a security review ticket created and approved before merge, and all new API endpoints must have corresponding automated security test cases committed in the same pull request. In each sprint planning session, the practitioner reviews the user stories in the upcoming sprint, identifies any that involve new data flows or trust boundaries, and adds a threat modelling story to that sprint's backlog. During the sprint retrospective, they review any security gate failures from the previous sprint and identify whether they indicate a systemic issue requiring a process change or a one-time finding. This routine — negotiated criteria, sprint-level threat modelling, retrospective review — keeps security embedded in the team's development rhythm without requiring a separate security review cycle.

Evaluating Methodology Risk for a Regulated Cloud Project

A Cloud Security Practitioner is asked to advise on methodology selection for a new cloud application that will process HIPAA-regulated health information for a US health system. They evaluate both Waterfall and Agile against the specific security constraints of the project. Waterfall's phase-gate documentation model aligns well with HIPAA's requirement for documented security controls and formal risk analysis, and the health system's auditors are experienced with reviewing Waterfall artefacts. Agile's iterative model offers faster delivery but requires the team to have mature security practices embedded in their sprint cycle — a condition the practitioner assesses the team does not yet meet, based on the team's history of deferring security work. The practitioner recommends a hybrid approach: a formal Waterfall requirements and design phase, producing the documented threat model and security architecture required for HIPAA compliance, followed by Agile sprints for implementation with automated security gates in the CI/CD pipeline. This decision gives the project the documentation compliance value of Waterfall and the delivery speed of Agile, while accounting for the team's current security maturity.

Best Practices

- **Shift security left — make security activities as early in the development cycle as possible.** Whether using Waterfall or Agile, the earlier a security issue is identified, the cheaper it is to fix. In Waterfall, this means conducting threat modelling during design rather than waiting for a penetration test. In Agile, this means including security requirements in sprint planning rather than reviewing security at project end.
- **Include security as a blocking criterion in the Agile definition of done.** A definition of done that includes security prevents the pattern of security being deferred sprint after sprint under deadline pressure. At minimum, configure at least one automated security gate as a CI pipeline blocker that cannot be bypassed, so the criterion is enforced mechanically rather than relying on team discipline alone.

- **In Waterfall projects, produce and store security artefacts at every phase exit.** Security artefacts — signed-off requirements, annotated architecture diagrams, penetration test reports — are the compliance evidence that Waterfall's documentation model is designed to produce. Incomplete artefacts undermine the compliance value of the entire methodology and leave the organisation without the audit trail it needs.
- **In Agile teams, conduct sprint-level threat modelling for any sprint introducing new functionality.** A single threat modelling session does not remain valid as Agile teams add new data flows, integrations, and trust boundaries sprint by sprint. Each sprint that introduces new functionality should include a focused threat modelling story, reviewing the new surface introduced and ensuring that security requirements are captured before implementation begins.
- **Start CI pipeline security tooling with narrow, high-confidence rules and expand gradually.** Alert fatigue from too many false positives causes teams to disable or ignore automated security tools. Configure a small set of high-confidence, high-severity rules first — blocking on OWASP Top 10 findings, critical CVEs in container images, and cloud storage public-access misconfigurations. Expand coverage once the team has established a triage process that keeps alert volume manageable.

Common Mistakes and Misconceptions

Misconception: Treating Agile's iterative speed as inherently incompatible with security, or treating Waterfall's documentation model as a substitute for security thinking.

Correct approach: Both methodologies can produce secure software and both can fail. The determining factor is whether security is built into the methodology — through Agile's definition of done and automated pipeline gates, or Waterfall's security phase exits and formal artefacts — not which methodology is chosen.

Misconception: In Agile teams, treating security as a sprint item that can be deferred when the team is under deadline pressure.

Correct approach: Security deferred once tends to be deferred repeatedly. The best practice is making security a blocking criterion in the definition of done and enforcing it through CI pipeline automation, so that deadline pressure does not override security requirements without an explicit, documented risk acceptance decision.

Misconception: In Waterfall projects, treating the formal security testing phase as the only opportunity for security involvement.

Correct approach: Security practitioners who appear only at the testing phase have missed every opportunity to influence design and implementation at a lower cost. The best practice is participating in each phase gate — requirements, design, implementation, testing, deployment — as an active reviewer with sign-off authority.

Misconception: Enabling all available CI pipeline security checks on day one, creating alert fatigue that causes the team to disable the tools.

Correct approach: Start with a narrow set of high-confidence, high-severity rules that produce actionable findings the team can realistically triage and remediate. Expand rule coverage incrementally as the team builds the triage process and capacity to act on findings.

table: Waterfall vs. Agile Security Comparison

Use this table to evaluate the security strengths, weaknesses, and best practices of each methodology when selecting or adapting a development process for a cloud application.

Dimension	Waterfall	Agile
Structure	Sequential phases with formal gates	Short iterative sprints with continuous delivery
Security Entry Point	Requirements phase; security gate at each phase exit	Product backlog; security embedded in each sprint
Primary Strength	Complete documentation and audit trail for compliance	Rapid detection and remediation of vulnerabilities via automation
Primary Weakness	Long gap between vulnerability introduction and discovery	Sprint pressure can cause security to be repeatedly deferred
Key Security Artefact	Phase-exit sign-offs, threat model, penetration test report	Definition of done, automated scan reports, sprint threat models
Best Fit	Regulated industries requiring formal compliance documentation	Cloud-native teams with mature DevSecOps tooling
Shift-Left Mechanism	Security gates before each phase begins	Security in backlog, sprint-level threat modelling, CI pipeline gates
Compliance Alignment	Well-aligned to PCI DSS, HIPAA, NIST RMF audit requirements	Requires additional documentation practices to satisfy auditors

How to use this: Many organisations use hybrid approaches: Waterfall requirements and design phases for documentation compliance, followed by Agile implementation sprints with automated security gates. Select the model that matches both the team's maturity and the regulatory environment.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

Q1. A development team is building a cloud application for a US Department of Defense subcontractor. The contracting agency requires a formal Authority to Operate (ATO), which demands documented security requirements, approved architecture, and a formal security assessment. Which SDLC methodology is most naturally aligned to these requirements, and why?

- A) A. Agile, because sprint-level threat modelling produces more current threat analysis than Waterfall design-phase reviews.
- B) B. Waterfall, because its sequential phase structure produces formal documented artefacts at each phase that directly support the ATO documentation requirements.
- C) C. Agile, because its continuous delivery model allows security issues to be identified and remediated faster.
- D) D. Waterfall, because it prevents developers from introducing new functionality without security review.

Answer: B. Waterfall, because its sequential phase structure produces formal documented artefacts at each phase that directly support the ATO documentation requirements.

Q2. An Agile development team has been deferring security-related user stories in their backlog for three consecutive sprints due to deadline pressure. The deferred items include input validation requirements and an outdated dependency update. Which of the following is the most effective structural control to prevent this pattern from recurring?

- A) A. Schedule a dedicated security sprint every quarter to address accumulated security debt.
- B) B. Include security criteria as blocking requirements in the team's definition of done, enforced by automated CI pipeline gates.
- C) C. Assign a security engineer to attend every sprint review and flag deferred security items.

D) D. Reduce sprint velocity targets to create capacity for security tasks.

Answer: B. Include security criteria as blocking requirements in the team's definition of done, enforced by automated CI pipeline gates.

Q3. Microsoft's early 2000s security transformation embedded security requirements and threat modelling into its development process rather than treating them as a post-development audit. What measurable outcome did this approach produce?

- A) A. Development velocity increased because security reviews no longer blocked releases.
- B) B. Critical vulnerability counts in shipped products decreased as security became embedded in the development rhythm.
- C) C. The company eliminated the need for penetration testing in subsequent product releases.
- D) D. Waterfall replaced Agile as the primary development methodology for all product teams.

Answer: B. Critical vulnerability counts in shipped products decreased as security became embedded in the development rhythm.

Q4. A Cloud Security Practitioner configures a CI/CD pipeline with 47 security rules, including checks for minor style violations and low-severity informational findings. Within two weeks, the development team has disabled the security scan stage because it generates too many alerts. What best practice was violated?

- A) A. Security scans should only run on the main branch, not on feature branches.
- B) B. CI pipeline security tooling should start with a narrow set of high-confidence, high-severity rules and expand coverage gradually once the team has a triage process.
- C) C. Security scan results should be reviewed by a dedicated security team rather than the development team.
- D) D. Informational findings should be promoted to warnings to ensure the team reviews them.

Answer: B. CI pipeline security tooling should start with a narrow set of high-confidence, high-severity rules and expand coverage gradually once the team has a triage process.

Q5. Which of the following most accurately describes the relationship between development methodology and security quality?

- A) A. Waterfall consistently produces more secure software than Agile because its formal gates prevent vulnerabilities from advancing to the next phase.
- B) B. Agile consistently produces more secure software than Waterfall because automated pipeline checks detect vulnerabilities faster than manual phase reviews.
- C) C. Either methodology can produce secure software or fail; the determining factor is whether security is built into the methodology from the beginning.
- D) D. Security quality is determined by the penetration testing phase, which both methodologies include.

Answer: C. Either methodology can produce secure software or fail; the determining factor is whether security is built into the methodology from the beginning.

Reflection Questions

1. Think about a development team you have worked with or observed. Did they use Waterfall, Agile, or a hybrid? At what points in their process did security receive attention? What would you change if you were designing their process today?

2. The lecture describes sprint pressure as a genuine risk in Agile — security tasks being deferred and rescheduled repeatedly. Have you observed this pattern in practice? What organisational dynamics drive it, and what would have to change to interrupt it?
3. Microsoft's SDL is now more than two decades old. What does the longevity of that approach tell you about the relationship between security culture and security practice? Is an SDL a process, a culture, or both?
4. If you were advising a start-up with a five-person development team building a HIPAA-regulated cloud application, would you recommend Waterfall, Agile, or a hybrid? What factors would most influence your recommendation?
5. Waterfall's documentation strength is also its weakness — the documentation can become an end in itself rather than evidence of security thinking. How would you distinguish a genuinely secure Waterfall project from one that has complete documentation but insecure code?

Key Takeaways

- Waterfall uses sequential phases with formal security gates at each phase exit, producing documentation artefacts that have direct compliance value in regulated industries.
- Agile uses short iterative sprints and embeds security through backlog management, sprint-level threat modelling, and automated security gates in the CI/CD pipeline.
- Waterfall's strength is documentation and auditability; its risk is the time gap between vulnerability introduction and discovery during formal testing.
- Agile's strength is rapid vulnerability detection through automation; its risk is sprint pressure causing security work to be repeatedly deferred.
- Including security as a blocking criterion in the Agile definition of done, enforced by CI pipeline automation, is the most effective structural control against security deferral.
- Neither methodology is inherently more secure. The determining factor is whether security is built into the process — through Waterfall's phase gates or Agile's sprint-embedded controls.
- Shifting security left — making security activities as early as possible in the development cycle — is the economically rational choice in both methodologies, because the cost of fixing a flaw increases by an order of magnitude for each phase it is deferred.

Recommended Next Actions

Put this lecture to work within the next seven days:

6. Review your team's current definition of done (or its equivalent). Identify whether any security criteria are included. If not, propose at least two specific, automatable security criteria to add — for example, a static analysis gate and a dependency CVE check — and present the proposal in the next sprint retrospective.
7. If your team uses a CI/CD pipeline, audit the security tools currently configured. Are they producing actionable findings that the team actually triages? If not, identify whether the problem is too many rules (alert fatigue) or too few. Adjust the rule configuration based on that diagnosis this week.
8. Identify one upcoming design review or architecture decision in your team's near-term roadmap. Propose adding a 30-minute STRIDE walkthrough to the design review agenda, and prepare a one-page data flow diagram to use as the basis for the discussion.
9. If your team uses Agile, review the last three sprint retrospectives for any pattern of security items being deferred. Document the pattern and propose a structural change — a pipeline gate, a definition of done criterion, or a policy — that would make the deferral visible and require explicit risk acceptance.